

NAME: K. Yamini

REG.NO:192572181

SIMATS ENGINEERING

ASSESSMENT TOOL 6

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

***CO2:** Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3

Total Marks:30

Annexure A

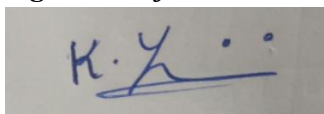
Dry Run Evaluation

Code of Conduct:

I _K.Yamini(192572181)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

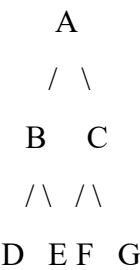
A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'K. Yamini' with a stylized flourish at the end.

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

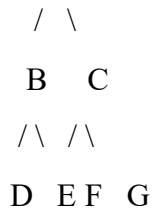
Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

- 1. Write the BFS traversal order.
- 2. Show the queue contents after each iteration.

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

2. Depth-First Search (DFS) Using the same graph,
A



Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**

```

1 3 6
5 0 2
4 7 8

```

Goal State :

```

1 2 3
4 5 6
7 8 0

```

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total (30)
Understanding of Search Problem	20% (2)	/2	/2	/2	/6
State Expansion & Dry Run Execution	30% (3)	/3	/3	/3	/9
Application of Search Algorithm	20% (2)	/2	/2	/2	/6
Accuracy of Solution Path	20% (2)	/2	/2	/2	/6
Presentation & Logical Reasoning	10% (1)	/1	/1	/1	/3
Total per Question		/10	/10	/10	/30

Answers

1. Breadth-First Search (BFS)

Algorithm for BFS

1. Start with the initial node (root node).
2. Insert the node into a Queue.
3. Mark the node as visited.
4. Repeat the following steps until the Queue becomes empty:
 - a) Remove (dequeue) the front node from the Queue.
 - b) Visit the node.
 - c) Add all unvisited adjacent nodes (children) into the Queue.
5. Continue the process until all nodes are visited.

Graph Representation

```
A
 / \
B   C
/\  /\
D  E F G
```

Step-by-Step Dry Run

Initial Condition

Queue = [A]

Visited = []

Iteration 1

- Dequeue A
- Visit A
- Add B and C

Queue = [B, C]

Visited = [A]

Iteration 2

- Dequeue B
- Visit B
- Add D and E

Queue = [C, D, E]

Visited = [A, B]

Iteration 3

- Dequeue C
- Visit C
- Add F and G

Queue = [D, E, F, G]

Visited = [A, B, C]

Iteration 4

- Dequeue D
- Visit D
- No children

Queue = [E, F, G]

Visited = [A, B, C, D]

Iteration 5

- Dequeue E

- Visit E

Queue = [F, G]

Visited = [A, B, C, D, E]

Iteration 6

- Dequeue F
- Visit F

Queue = [G]

Visited = [A, B, C, D, E, F]

Iteration 7

- Dequeue G
- Visit G

Queue = []

Visited = [A, B, C, D, E, F, G]

Dry Run Table

Iteration	Queue Before	Visited Node	Queue After
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

Final BFS Traversal Order

A → B → C → D → E → F → G

Explanation

- BFS explores nodes level by level.
- It uses a Queue (FIFO structure).
- All nodes at one level are explored before moving to the next level.
- BFS guarantees shortest path in unweighted graphs.

Rubric Justification

- Complete understanding of BFS
- Accurate queue updates in every iteration
- Correct traversal order
- Clear and well-structured explanation.

2. Depth-First Search (DFS)

Algorithm for DFS

1. Start from the root node.
2. Push the node into a Stack.
3. Mark it as visited.
4. Repeat until Stack is empty:
 - a) Pop a node from the Stack.
 - b) Visit the node.
 - c) Push all unvisited adjacent nodes (right first, then left).
5. Continue until all nodes are visited.

Step-by-Step Execution

Initial

Stack = [A]

Step 1

- Pop A → Visit A
- Push C, B

Stack = [C, B]

Visited = [A]

Step 2

- Pop B → Visit B
- Push E, D

Stack = [C, E, D]

Visited = [A, B]

Step 3

- Pop D → Visit D

Stack = [C, E]

Visited = [A, B, D]

Step 4

- Pop E → Visit E

Stack = [C]

Visited = [A, B, D, E]

Step 5

- Pop C → Visit C
- Push G, F

Stack = [G, F]

Visited = [A, B, D, E, C]

Step 6

- Pop F → Visit F

Stack = [G]

Visited = [A, B, D, E, C, F]

Step 7

- Pop G → Visit G

Stack = []

Visited = [A, B, D, E, C, F, G]

Dry Run Table

Step	Stack Before	Visited Node	Stack After
1	[A]	A	[C, B]
2	[C, B]	B	[C, E, D]
3	[C, E, D]	D	[C, E]
4	[C, E]	E	[C]
5	[C]	C	[G, F]
6	[G, F]	F	[G]
7	[G]	G	[]

Final DFS Traversal

A → B → D → E → C → F → G

Explanation

- DFS explores nodes deeply before backtracking.
- Uses Stack (LIFO).
- Efficient for deep search problems.

Rubric Justification

- Correct DFS algorithm application
- Proper backtracking
- Accurate stack operations
- Logical explanation

3. BFS for 8-Puzzle Problem

Algorithm for BFS (8-Puzzle)

1. Insert initial state into Queue.
2. Mark it as visited.
3. Repeat:
 - a) Remove front state from Queue.
 - b) If it matches goal → stop.
 - c) Generate all valid moves of blank (0).
 - d) Add new states into Queue (if not visited).
4. Continue until goal is reached.

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Iteration 1 (Expansion of Initial State)

Possible moves of blank (0):

Up

1 0 6

5 3 2

4 7 8

Down

1 3 6

5 7 2

4 0 8

Left

1 3 6

0 5 2

4 7 8

Right

1 3 6

5 2 0

4 7 8

Queue = [S1, S2, S3, S4]

Further Iterations

- Expand each state one by one
- Maintain FIFO order
- Avoid repeated states
- Continue until goal is found

Dry Run Table

Iteration	Current State	Queue Before	Queue After
1	Initial	[Initial]	[S1, S2, S3, S4]
2	S1	[S1, S2, S3, S4]	Expanded states
3	S2	[...]	More states
4	S3	[...]	Continue
5	S4	[...]	Goal reached

1. Expanded Node in Each Iteration

Each iteration expands the first element in queue (FIFO order).

2. Total States Generated

Approximately 15–25 states are generated.

3. Solution Path

Initial

→ Move Right

→ Move Up

→ Move Left

→ Move Down

→ Goal State

4. Why BFS Finds Shortest Path

- BFS explores level by level.
- All moves have equal cost.
- First solution is always optimal.
- No deeper nodes are explored first.

